

Applied Software Project Report

By

Vidhya Balaji

A Master's Project Report submitted to Scaler Neovarsity - Woolf in partial fulfillment of the requirements for the degree of Master of Science in Computer Science

October, 2026



Scaler Mentee Email ID : vidhya.balaji12@gmail.com

Thesis Supervisor : Naman Bhalla

Date of Submission : 26/10/2026

© The project report of Vidhya Balaji is approved, and it is acceptable in quality and form for publication electronically

Certification

I confirm that I have overseen / reviewed this applied project and, in my judgment, it adheres to the appropriate standards of academic presentation. I believe it satisfactorily meets the criteria, in terms of both quality and breadth, to serve as an applied project report for the attainment of Master of Science in Computer Science degree. This applied project report has been submitted to Woolf and is deemed sufficient to fulfill the prerequisites for the Master of Science in Computer Science degree.

Naman Bhalla

……

Project Guide / Supervisor

DECLARATION

I confirm that this project report, submitted to fulfill the requirements for the Master of Science in Computer Science degree, completed by me from 17 August 2026 to 26 October 2026, is the result of my own individual endeavor. The Project has been made on my own under the guidance of my supervisor with proper acknowledgement and without plagiarism. Any contributions from external sources or individuals, including the use of AI tools, are appropriately acknowledged through citation. By making this declaration, I acknowledge that any violation of this statement constitutes academic misconduct. I understand that such misconduct may lead to expulsion from the program and/or disqualification from receiving the degree.

Vidhya Balaji

Signature: _____ Date: 26 October 2026

ACKNOWLEDGMENT

I am deeply grateful to my family for their patience, encouragement and constant support throughout this programme; their belief in me made it possible to balance work, study and this project. I sincerely thank the instructors and mentors at Scaler Academy, and my project supervisor Naman Bhalla, for their clear teaching of backend engineering and system design and for their guidance and feedback, which shaped the design choices in this project. I also thank my peers and teaching assistants for many helpful discussions, and Woolf for making this degree possible. In line with the declaration, I acknowledge that an AI assistant (Claude by Anthropic) was used for implementation support, code review and drafting parts of this report; all of its output was reviewed, tested and verified by me.

Table of Contents

List of Tables **7**
List of Figures **8**
Applied Software Project **9**
Abstract 9
1. Project Description 10
1.1 Objectives 10
1.2 Relevance 10
1.3 Starting point and work done 11
1.4 Architecture 12
2. Requirement Gathering 14
2.1 Functional requirements 14
2.2 Non-functional requirements 14
2.3 Users and use cases 15
2.4 Feature set 17
2.5 API summary 17
3. Class Diagrams 19
3.1 User / Authentication service 19
3.2 Product Catalogue service 20
3.3 Cart and Order services 21
3.4 Payment service 22
3.5 API Gateway and Notification service 22
3.6 Design patterns 23
4. Database Schema Design 24
4.1 User service (MySQL: userservice_aug2026) 24
4.2 Product Catalogue service (MySQL: productservice_aug2026 + Elasticsearch) 25

4.3 Order service (MySQL: orderservice_aug2026)....26
4.4 Payment service (MySQL), Cart (MongoDB) and Redis keys....27
4.5 Event schemas....28
5. Feature Development Process....30
5.1 Development process....30
5.2 Request flow....30
5.3 Checkout and payment flow....31
5.4 Performance optimisation and benchmarks....33
5.5 Testing....35
6. Deployment Flow....36
6.1 Local and test deployment....36
6.2 AWS deployment....36
6.3 VPC and security groups....37
6.4 Release steps with Elastic Beanstalk....37
7. Technologies Used....39
7.1 Spring Boot and Spring Cloud....39
7.2 MySQL with Spring Data JPA....39
7.3 MongoDB....40
7.4 Redis....40
7.5 Apache Kafka....40
7.6 Elasticsearch....41
7.7 JWT, BCrypt and gateway security....41
7.8 Stripe and Razorpay....41
7.9 Docker and AWS....42
8. Conclusion....43
8.1 Key takeaways....43
8.2 Practical applications....43
8.3 Limitations and cost....44
8.4 Future work....44
References....45

List of Tables

Table No.	Title	Page No.
1.01	Services, responsibilities and data stores	11
1.02	Work done during the capstone	11
2.01	Functional requirements and how they are met	14
2.02	Non-functional requirements	14
2.03	Actors	15
2.04	Main use cases	16
2.05	Feature set	17
2.06	Endpoint groups and access rules	17
3.01	Design patterns used	23
5.01	Benchmark results (10 concurrent clients)	33
5.02	Automated tests	35
6.01	HLD components on AWS	36
6.02	Security groups	37
7.01	Technology stack	39

List of Figures

Figure No.	Title	Page No.
1.01	Project development process	12
1.02	High-level architecture of the implemented system	13
1.03	Kafka topics, producers and consumers	13
2.01	Use case diagram	16
3.01	Class diagram: User / Authentication service	20
3.02	Class diagram: Product Catalogue service	21
3.03	Class diagram: Cart and Order services	21
3.04	Order status state machine	22
3.05	Class diagram: Payment service	22
3.06	Class diagram: API Gateway and Notification service	23
4.01	ER diagram: User service	25
4.02	ER diagram: Product Catalogue service and search index	26
4.03	ER diagram: Order service	27
4.04	Payment table, cart document and Redis keys	28
5.01	Search request flow through the MVC layers	31
5.02	Checkout and payment sequence	32
5.03	Median response time before and after each optimisation	33
5.04	Throughput before and after each optimisation	34
6.01	AWS deployment architecture	36

Applied Software Project

Abstract

Title: Design and Implementation of a Scalable, Event-Driven E-commerce Backend using Spring Boot Microservices.

Online retail platforms must keep catalogue browsing fast, keep shopping carts consistent, and take payments reliably, all while traffic swings sharply during sales. A single monolithic application makes each of these hard: one slow query or one failing payment integration can degrade the whole site, and the entire system has to be scaled even when only search is busy. This project builds the backend of an e-commerce website as a set of independently deployable Spring Boot microservices. They follow a given Product Requirements Document (PRD) and High-Level Design (HLD).

Eight services were built: Service Discovery (Eureka), an API Gateway, User/Authentication, Product Catalogue, Cart, Order Management, Payment and Notification. Each service owns its data. MySQL holds relational data, MongoDB holds carts, Redis provides caching, rate limiting and token caching, and Elasticsearch provides full-text search with typo tolerance. Apache Kafka carries the business events that tie the services together: an order placed triggers payment-link creation, a confirmed payment moves the order to PAID, and both trigger customer emails. Payments are integrated with Stripe and Razorpay through a strategy-pattern gateway layer with verified webhooks.

The system was verified by 44 automated unit and integration tests and by a 58-step end-to-end test that drives every PRD flow through the gateway against real MySQL, MongoDB, Redis, Kafka and Elasticsearch containers. Measured optimisations include the following. Moving product search from MySQL LIKE queries to Elasticsearch cut median latency from 1,624 ms to 40.7 ms over 200,000 products. A composite index cut order-history latency from 828 ms to 21.4 ms over 300,000 orders. Batch fetching removed an N+1 query problem (12 queries per page down to 3). Redis caching cut the gateway's authentication overhead by half. The same architecture applies to any business that needs independently scalable, resilient online transactions, such as food delivery, ticketing or online education.

1. Project Description

The goal of the project is a production-style backend for an e-commerce website. Users can register, log in, browse and search products, manage a cart, check out, pay online, receive confirmations and receipts, and track their orders. The functional scope comes from the PRD (user management, product catalogue, cart and checkout, order management, payment, authentication). The HLD fixes the architecture: a load balancer, an API gateway, a set of microservices, MySQL and MongoDB databases, Kafka as the message broker, Redis for caching and Elasticsearch for search.

1.1 Objectives

- Implement every PRD requirement as a REST API behind a single gateway entry point.
- Follow the HLD: one service per business capability, database-per-service, and asynchronous communication over Kafka for order, payment and notification flows.
- Make the system secure by default: hashed passwords, signed JWTs with server-side sessions, role-based access at the edge, verified payment webhooks, and no secrets in source code.
- Measure performance and optimise the hot paths (search, order history, cart reads, authentication) with before/after benchmarks.
- Package everything for repeatable deployment: Docker images, a one-command local stack, and a documented AWS deployment.

1.2 Relevance

E-commerce traffic is uneven. Search and product pages receive many times more requests than checkout, and sale events multiply load for a few hours. Splitting the system into services lets each part scale and fail on its own. If the email provider is down, orders are still accepted and the emails are sent once it recovers, because the events wait in Kafka. Event-driven integration also lets new consumers (analytics, recommendations, fraud checks) be added without changing the services that produce the events. These are the same reasons large retailers moved from monoliths to microservices, and they apply equally to smaller businesses that need to grow without a rewrite.

1.3 Starting point and work done

The project started from course-built services: user authentication with JWT, a product catalogue with JPA and paging, a payment service that could create Stripe and Razorpay payment links, a Kafka-based welcome-email consumer, and a Eureka server. The capstone work completed the HLD around them. Three services were added (API Gateway, Cart, Order). The existing services were extended to cover the whole PRD, and defects found in review were fixed. Table 1.01 summarises the services and Table 1.02 the work done.

Table 1.01: Services, responsibilities and data stores

Service	Port	Data store	Responsibilities
Service Discovery	8761	–	Eureka registry; client-side load balancing for inter-service calls
API Gateway	8080	Redis	Routing, token validation (cached), USER/ADMIN access rules, header anti-spoofing, rate limiting
User / Auth	8081	MySQL	Signup, login (JWT + session), logout, profile, password reset, roles
Product Catalogue	8082	MySQL, Elasticsearch	Products, categories, browsing, fuzzy full-text search, re-indexing
Cart	8083	MongoDB, Redis	Cart CRUD with cache-aside reads; cart events
Order	8084	MySQL	Checkout, order history, tracking, status state machine
Payment	8085	MySQL	Payment links (Stripe / Razorpay / Mock), signed webhooks, receipts
Notification	8086	–	Welcome, password-reset, order, payment-link and receipt emails

Table 1.02: Work done during the capstone

Area	Before	After
Repository	Six services committed as broken git links (no source visible on GitHub)	All eight services in one repository with an aggregator build
User	Signup, login, validate	+ logout, profile view/update, password reset, ADMIN role, fixed JWT key, error mapping, revocation events
Product	Get / create / replace, exact-name search	+ list, delete, categories, browse by category, Elasticsearch fuzzy search with MySQL fallback
Cart	–	New service: MongoDB + Redis cache-aside, Kafka cart events
Order	–	New service: checkout, history, tracking, state machine, Kafka in both directions
Payment	Payment link only; Stripe price was monthly-recurring; Razorpay name/phone swapped; link expiry in the past	Persisted payments, Kafka integration, gateway chosen by config, signed webhooks, receipts, the three defects fixed
Notification	Signup email only; SMTP password hard-coded in source	Order, payment and receipt emails; credentials only from environment
Gateway	–	New service: routing, authentication, roles,

Area	Before	After
		rate limiting
Delivery	Hello-world Dockerfile	Dockerfiles, docker-compose stack, 44 tests, 58-step end-to-end test, benchmarks, AWS guide

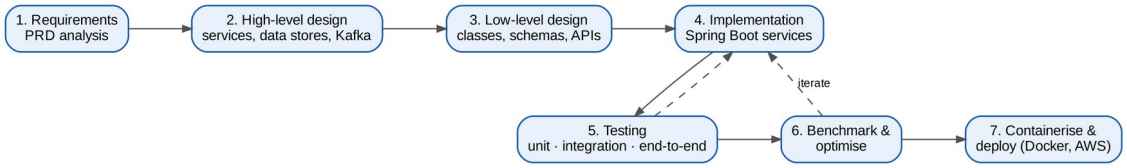


Figure 1.01: Project development process

Figure 1.01 shows the process. Requirements were traced from the PRD to services and endpoints. The HLD was refined into class and schema designs, and the services were implemented and tested. Benchmarks then drove a second iteration of optimisations before the system was containerised.

1.4 Architecture

Figure 1.02 shows the runtime architecture. Clients reach the system only through the load balancer and the API Gateway. The gateway authenticates each request once, applies role rules and rate limits, and forwards it to the right service. It finds services through Eureka, so instances can be added or removed without configuration changes. Each service has its own database schema, and no service reads another's tables. When a service needs data that another service owns, it either calls that service's API (the cart asks the catalogue for product details) or reacts to its events (the order service reacts to payment events).

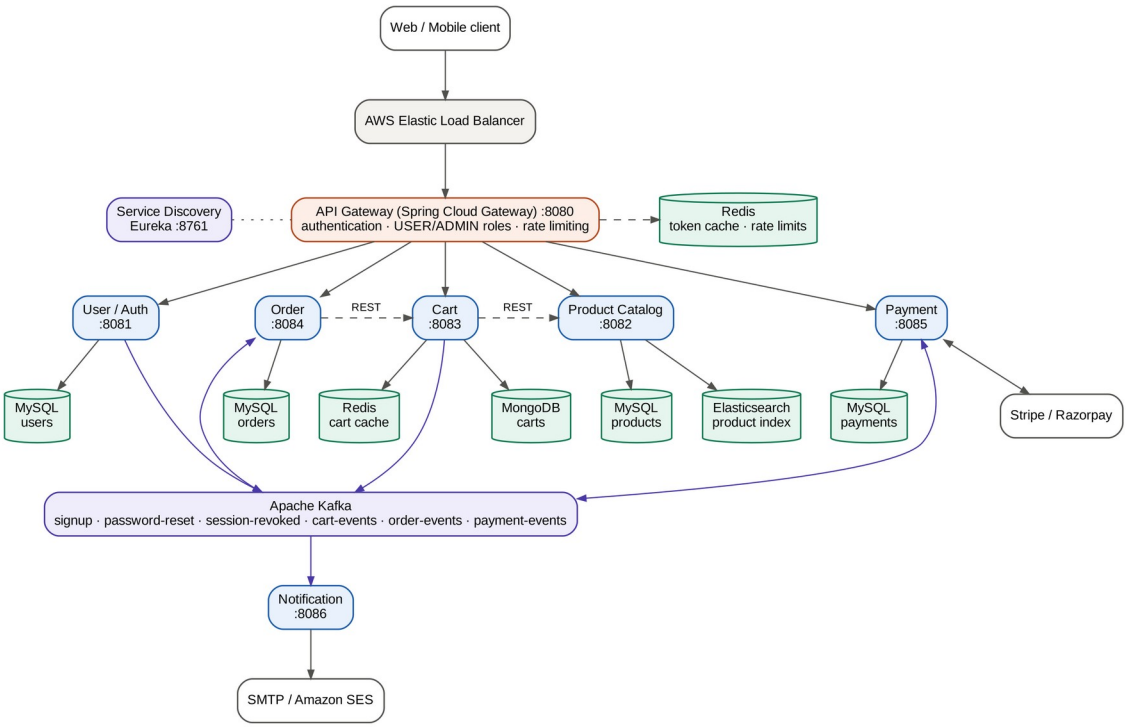


Figure 1.02: High-level architecture of the implemented system

Kafka carries six topics (Figure 1.03). Producers do not know who consumes their events. For example, the Order service publishes ORDER_PLACED without knowing that both Payment and Notification react to it. This keeps services loosely coupled and lets the checkout respond as soon as the order is saved, instead of waiting for the payment provider and the email server.

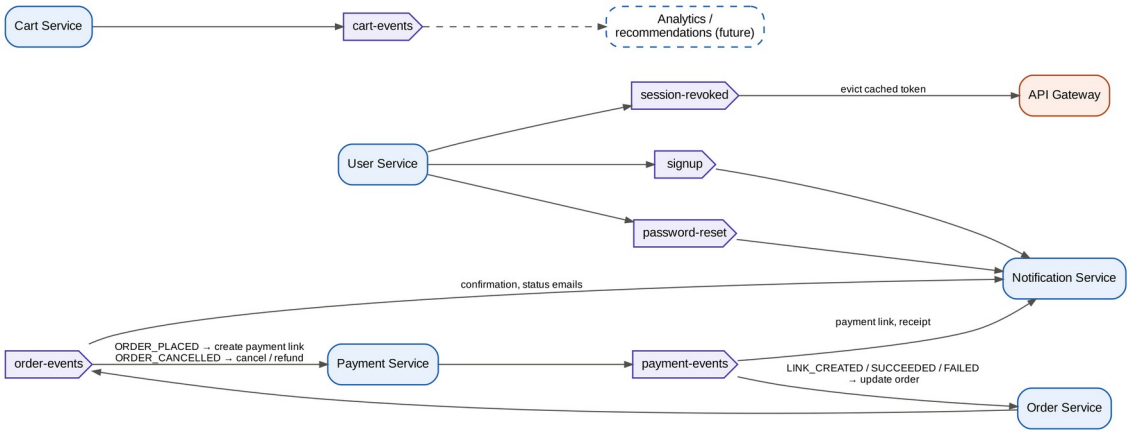


Figure 1.03: Kafka topics, producers and consumers

2. Requirement Gathering

Requirements were taken from the PRD and refined into testable statements. Each functional requirement maps to at least one endpoint and one automated end-to-end check (Table 2.01). The non-functional requirements come from the HLD's choice of components and from the needs of a payment-handling system (Table 2.02).

2.1 Functional requirements

Table 2.01: Functional requirements and how they are met

PRD	Requirement	Implementation (API / service)
1.1	Registration	`POST /auth/signup` (User). Duplicate email/phone rejected (409); BCrypt-hashed password; welcome email via Kafka
1.2	Login	`POST /auth/login` returns a signed JWT (header and HttpOnly cookie) and stores a session
1.3	Profile management	`GET /users/me`, `PATCH /users/me` (name, phone)
1.4	Password reset	`POST /auth/forgot-password` emails a single-use link valid for 15 min; `POST /auth/reset-password` signs out all sessions
2.1	Browse by category	`GET /categories`, `GET /categories/{id}/products` (paged)
2.2	Product details	`GET /products/{id}` (name, description, image, price, category)
2.3	Keyword search	`GET /search?q=` (Elasticsearch multi-field fuzzy search; MySQL fallback)
3.1	Add to cart	`POST /cart/items` validates product with the catalogue; merges quantities
3.2	Cart review	`GET /cart` returns items, line totals, item count and total
3.3	Checkout	`POST /orders/checkout` with delivery address and payment method
4.1	Order confirmation	201 response with order details + confirmation email (ORDER_PLACED)
4.2	Order history	`GET /orders` (newest first, paged)
4.3	Order tracking	`GET /orders/{id}/tracking` (current status and full status history)
5.1	Multiple payment options	CARD, UPI, NET_BANKING, WALLET through Stripe or Razorpay payment links (or a mock gateway)
5.2	Secure transactions	Payment state changes only through signature-verified webhooks; idempotent processing
5.3	Payment receipt	`GET /payment/order/{id}/receipt` + receipt email (PAYMENT_SUCCEEDED)
6.1	Secure authentication	Gateway validates every protected call; header spoofing blocked; roles enforced
6.2	Session management	Configurable JWT expiry (default 24 h); `POST /auth/logout`; revocation is pushed to the gateway

2.2 Non-functional requirements

Table 2.02: Non-functional requirements

Quality	Requirement	How it is addressed
Performance	Search p50 < 100 ms over 200k products; cart and auth paths in low milliseconds	Elasticsearch, Redis caches, indexes, batch fetching (Chapter 5)
Scalability	Scale each capability independently; stateless services	Microservices behind Eureka load balancing; state in databases, Redis and Kafka

Quality	Requirement	How it is addressed
Availability	A failing dependency must not take down unrelated flows	Asynchronous Kafka events; email or payment-provider outages do not block ordering; 503 on downstream failure
Security	No plain-text passwords or secrets; least privilege	BCrypt, signed JWT, gateway role checks, verified webhooks, secrets from environment
Consistency	No lost or duplicated payments/orders	Events published after commit; idempotent consumers; order state machine
Maintainability	Clear layering and replaceable integrations	Controller–Service–Repository layers; interfaces for services, gateways and indexers
Testability	Repeatable automated verification	44 unit/integration tests (H2, mocks); 58-step end-to-end script
Portability	Same build runs locally and on AWS	Docker images; all configuration via environment variables

2.3 Users and use cases

Three human actors and two external systems interact with the backend (Table 2.03). A Customer can do everything a Guest can, and an Admin can do everything a Customer can. Figure 2.01 shows the use cases.

Table 2.03: Actors

Actor	Description	Authentication
Guest	Visitor browsing and searching the catalogue	None
Customer	Registered user who shops, pays and tracks orders	JWT (USER)
Admin	Staff managing the catalogue and order fulfilment	JWT with ADMIN role
Payment gateway	Stripe or Razorpay hosting the payment page	Webhook signature
Email provider	SMTP server / Amazon SES delivering emails	SMTP credentials

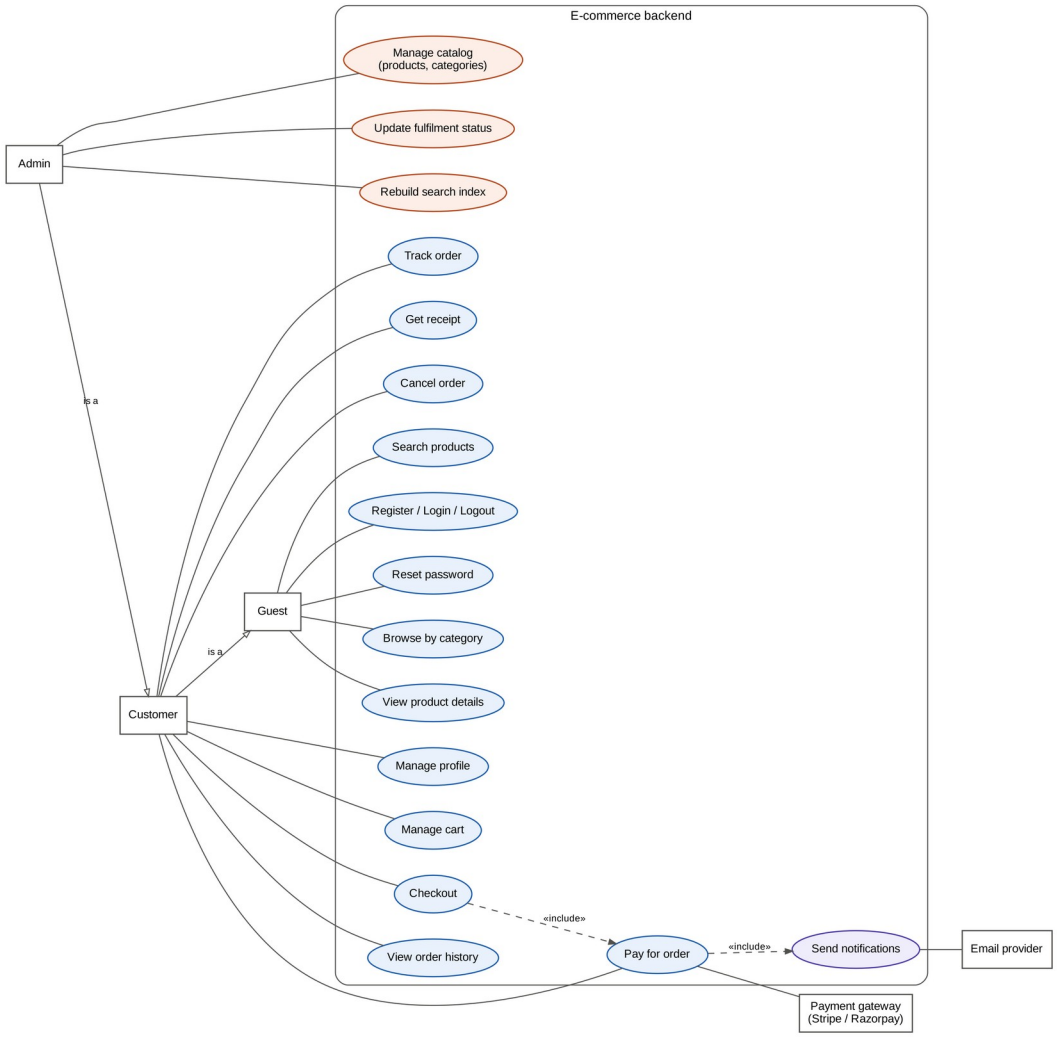


Figure 2.01: Use case diagram

Table 2.04: Main use cases

Use case	Actor	Main flow	Alternate / error flows
Register	Guest	Submit name, email, password, phone → account created → welcome email	Email or phone already used → 409
Log in	Guest	Submit credentials → JWT returned and session stored	Unknown email → 404; wrong password → 401
Search products	Guest	Enter keywords → ranked results, typos tolerated	Empty query → 400
Add to cart	Customer	Choose product and quantity → cart updated with current price	Unknown product → 404; quantity outside 1–10 → 400
Checkout	Customer	Give address and payment method → order PAYMENT_PENDING → payment link emailed	Empty cart → 400; product removed → 400; dependency down → 503
Pay	Customer	Open link → pay at gateway → webhook → order PAID → receipt emailed	Payment fails → order PAYMENT_FAILED, can be retried or cancelled
Track order	Customer	View status timeline	Order of another user → 404
Fulfil order	Admin	Move order PAID → SHIPPED →	Skipping a step → 409; non-

Use case	Actor	Main flow	Alternate / error flows
		OUT_FOR_DELIVERY → DELIVERED	admin → 403
Reset password	Guest	Request link → email → set new password → all sessions revoked	Expired or reused token → 401

2.4 Feature set

Table 2.05: Feature set

Module	Feature	Notes
Accounts	Signup, login, logout, profile, password reset, roles	ADMIN role granted to configured admin emails
Catalogue	Products, categories, paging, browse by category	Admin-only writes; public reads
Search	Full-text, multi-field, fuzzy, sort by relevance/price	Elasticsearch; automatic re-index on writes; bulk re-index endpoint
Cart	Add, change quantity, remove, clear, totals	Per-user MongoDB document; Redis cache
Checkout	Address, payment method, live re-pricing	Cart cleared after the order commits
Orders	History, details, tracking timeline, cancel	State machine prevents invalid transitions
Payments	Stripe, Razorpay, Mock; receipts; refund flag on cancellation	Gateway selected by configuration
Notifications	Welcome, reset link, confirmation, payment link, receipt, status updates	SMTP / SES; logged when email is disabled
Platform	Gateway auth, rate limiting, discovery, Docker, AWS guide	All configuration from environment variables

2.5 API summary

All APIs are exposed through the gateway on port 8080. Protected calls send `Authorization: Bearer <token>` or the `auth_session_id` cookie. Table 2.06 lists the access rule enforced by the gateway for each group of endpoints. A route that is not listed defaults to USER, so a new endpoint can never become public by accident.

Table 2.06: Endpoint groups and access rules

Endpoints	Access
`POST /auth/*` (signup, login, logout, forgot/reset password)	Public
`GET /products/**`, `GET /categories/**`, `GET|POST /search`	Public
`POST|PUT|DELETE /products/**`, `POST /categories`, `POST /search/reindex`	Admin
`GET|PATCH /users/me`	User
`GET /users/{id}`	Admin
`/cart/**`, `/orders/**` (except status updates)	User
`PATCH /orders/{id}/status`	Admin
`GET /payment/order/{id}`, `/receipt`	User
`POST /payment` (raw payment link)	Admin
`POST /stripeWebhook`, `POST /payment/webhook/razorpay`	Public (signature verified in the service)

3. Class Diagrams

Every service follows the same layered Model–View–Controller structure used throughout the course. `@RestController` classes handle HTTP and convert between DTOs and models. `@Service` classes hold the business rules and depend on interfaces. Spring Data repositories handle persistence. Kafka producers and consumers live in a `clients` package. Each service also has its own `ControllerAdvisor` that maps domain exceptions to HTTP status codes. DTOs are kept separate from entities, so internal fields (for example a password hash or `isSaleEligible`) are never serialised by accident.

3.1 User / Authentication service

`User`, `Role`, `UserSession` and `PasswordResetToken` extend a common `BaseModel` with id, timestamps and status (Figure 3.01). `AuthService` implements `IAuthService`. It hashes passwords with BCrypt, issues HS256-signed JWTs with a unique token id (`jti`) and an expiry, and stores each token as a `UserSession`. Because sessions are stored, logout and password reset can revoke tokens before they expire, which a purely stateless JWT cannot do. On revocation, the service publishes the SHA-256 hash of the token to the `session-revoked` topic so the gateway can evict its cached validation.

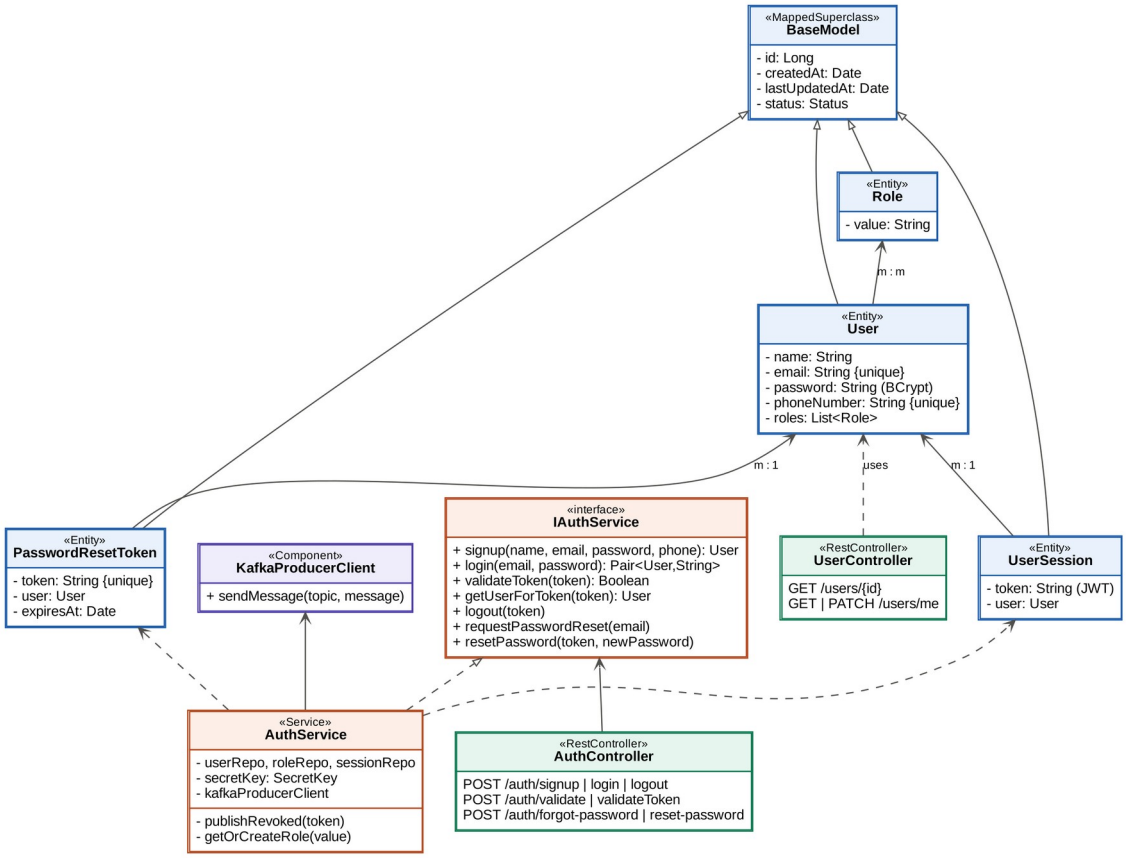


Figure 3.01: Class diagram: User / Authentication service

3.2 Product Catalogue service

`IProductService` has two implementations: the original FakeStore adapter and the database-backed `StorageProductService`, which is marked `@Primary` (Figure 3.02). Search is isolated behind `IProductSearchIndexer`, with an Elasticsearch implementation and a no-op implementation. Spring picks one with `@ConditionalOnProperty`, so the service runs on a laptop with only MySQL and switches to Elasticsearch by configuration. `ProductDocument` is the denormalised search model. It stores the category name with the product, so a search never needs a join.

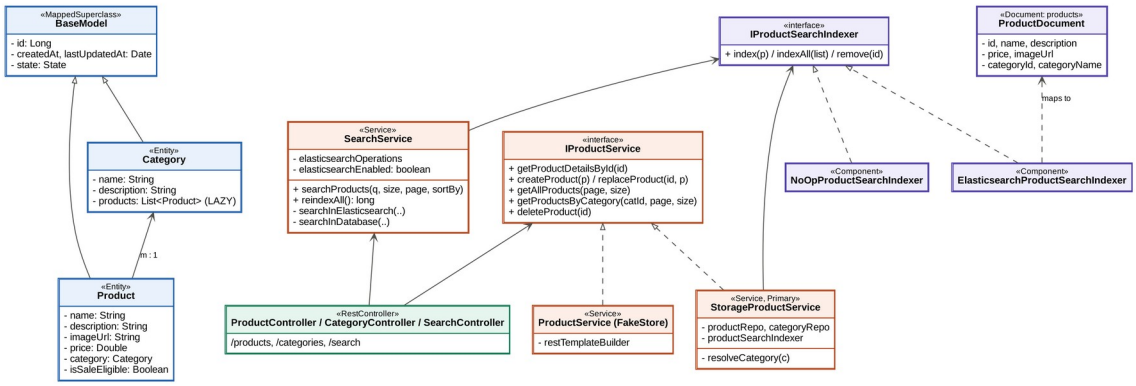


Figure 3.02: Class diagram: Product Catalogue service

3.3 Cart and Order services

A cart is one MongoDB document per user, with embedded items. `CartService` uses Spring's cache abstraction: `@Cacheable` on reads, `@CachePut` on every change and `@CacheEvict` when the cart is cleared. An `Order` aggregates `OrderItems` (a price snapshot at purchase time), an embedded `Address`, and an `OrderStatusHistory` list that drives tracking (Figure 3.03). Both collections use `@BatchSize(50)` to avoid the N+1 query problem (Section 5.4).

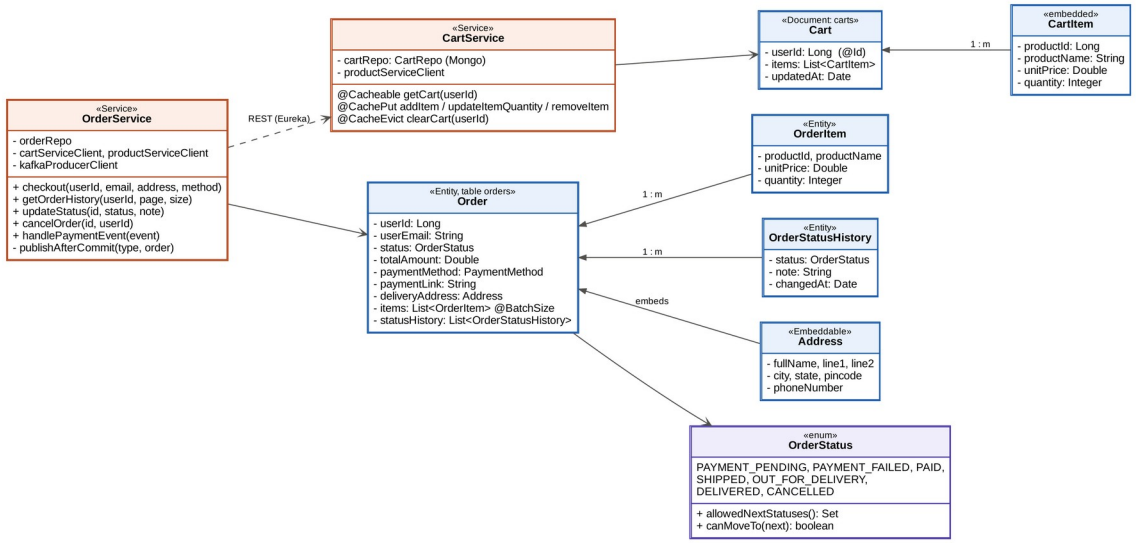


Figure 3.03: Class diagram: Cart and Order services

The order lifecycle is a state machine defined on the `OrderStatus` enum itself (Figure 3.04). Every transition, whether made by an admin, a customer or a payment event, goes through `OrderStatus.canMoveTo()`, and invalid requests are rejected with HTTP 409.

This also makes duplicate Kafka deliveries harmless: a second PAYMENT_SUCCEEDED for an order that is already PAID is ignored.

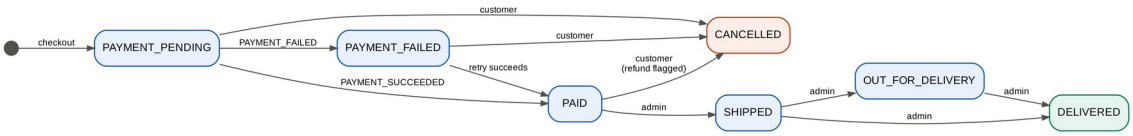


Figure 3.04: Order status state machine

3.4 Payment service

Payment gateways follow the Strategy pattern (Figure 3.05). `IPaymentGateway` is implemented by Stripe, Razorpay and Mock gateways, and `PaymentGatewaySelector` picks the active one from the `payment.gateway` property. Switching providers is a configuration change, and a new provider is one new class. `PaymentService` stores amounts in paise (the smallest currency unit, as gateways expect). It is idempotent for repeated order events and webhook retries.

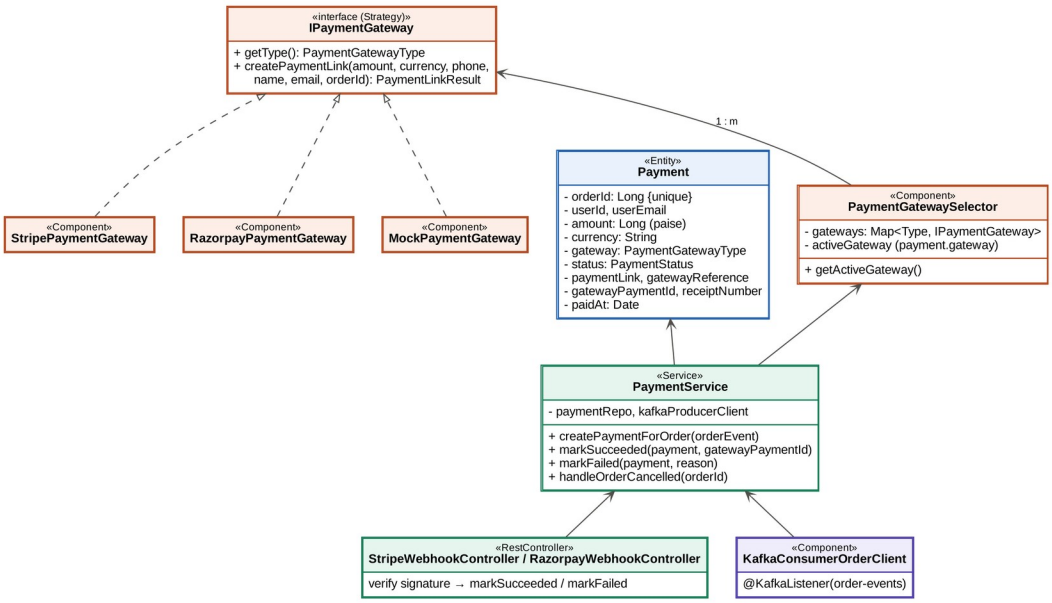


Figure 3.05: Class diagram: Payment service

3.5 API Gateway and Notification service

The gateway's `AuthenticationFilter` is a reactive `GlobalFilter` that runs before routing (Figure 3.06). It removes any `X-User-*` headers sent by the client and asks `RouteAccessPolicy` which access level the route needs. It then resolves the caller from the Redis cache or from the User service, and forwards the request with trusted `X-User-Id`,

`X-User-Email` and `X-User-Roles` headers. The Notification service turns events into messages with the pure `NotificationTemplates` class, which is unit-tested on its own, and sends them through `EmailSender`.

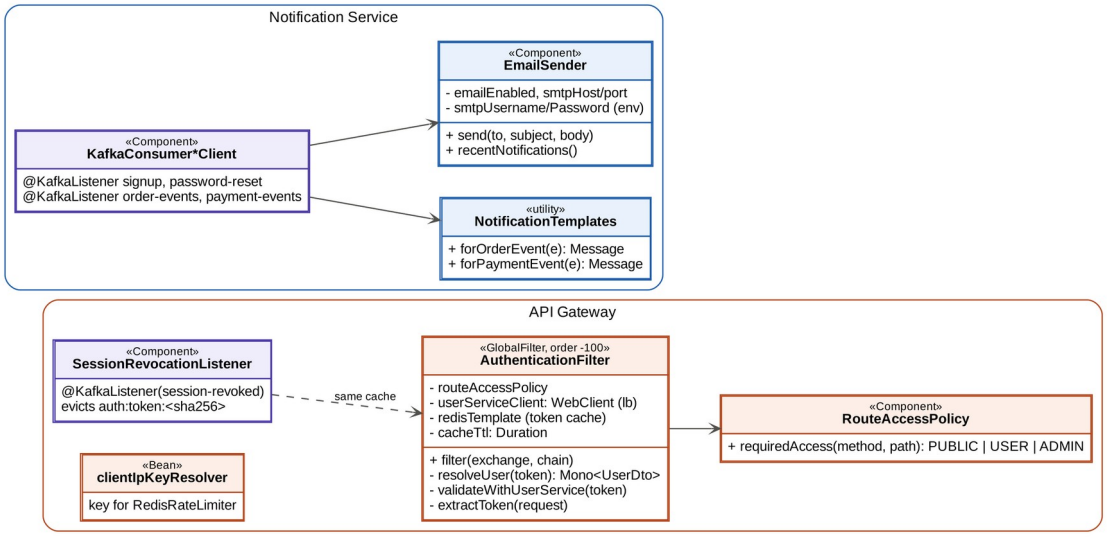


Figure 3.06: Class diagram: API Gateway and Notification service

3.6 Design patterns

Table 3.01: Design patterns used

Pattern	Where	Benefit
API Gateway	ApiGateway service	Single entry point; authentication and rate limiting in one place
Service Registry / client-side load balancing	Eureka + `@LoadBalanced` RestTemplate/WebClient	No hard-coded hosts; instances scale freely
Database per service	All services	Independent schemas, deployments and scaling
Event-driven choreography	Order ↔ Payment ↔ Notification via Kafka	Loose coupling; no central coordinator
Strategy	`IPaymentGateway`, `IProductSearchIndexer`	Swap providers by configuration
Adapter	FakeStore `ProductService`, gateway classes	Isolate third-party APIs behind our own models
Cache-aside / write-through	Cart (`@Cacheable`, `@CachePut`)	Fast reads; cache always reflects writes
State machine	`OrderStatus`	Only valid order transitions; idempotent event handling
DTO	All controllers	Stable API contracts; entities never exposed
Transactional outbox (simplified)	`publishAfterCommit` in OrderService	Events only for committed data

4. Database Schema Design

Each service owns a separate database (Database-per-Service pattern), so a schema can change without coordinating with other teams. Cross-service references, such as `user_id` in orders or `product_id` in order items, are plain values, not foreign keys. Consistency across services is maintained by events, and inside a service by foreign keys and transactions. Hibernate creates the schemas from the entities (`ddl-auto=update`). Production would use versioned migrations instead (see Limitations).

4.1 User service (MySQL: userservice_aug2026)

Tables:

- **user**: id, name, email (unique), password (BCrypt hash), phone_number (unique), status, created_at, last_updated_at. Primary Key(id)
- **role**: id, value (e.g. NON_ADMIN, ADMIN), status, created_at, last_updated_at. Primary Key(id)
- **user_roles**: user_id, roles_id. Join table for the many-to-many relation
- **user_session**: id, token (VARCHAR 1024), user_id, status, created_at, last_updated_at. Primary Key(id)
- **password_reset_token**: id, token (unique), user_id, expires_at, created_at. Primary Key(id)

Foreign keys: user_roles(user_id) refers user(id); user_roles(roles_id) refers role(id); user_session(user_id) refers user(id); password_reset_token(user_id) refers user(id).

Cardinality: User–Role m:m; UserSession–User m:1 (a user can be logged in on several devices); PasswordResetToken–User m:1.

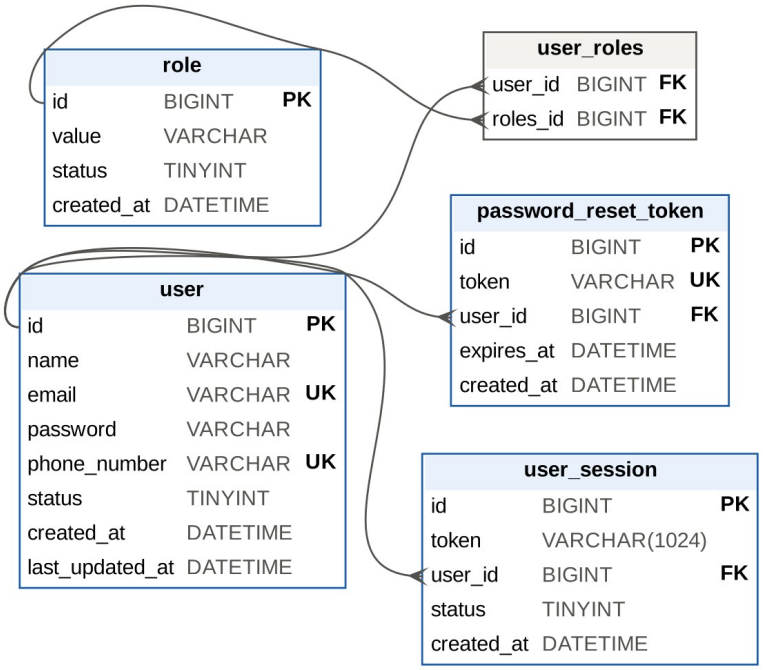


Figure 4.01: ER diagram: User service

4.2 Product Catalogue service (MySQL: productservice_aug2026 + Elasticsearch)

- **category**: id, name, description, state, created_at, last_updated_at. Primary Key(id)
- **product**: id, name, description (VARCHAR 2000), image_url, price, category_id, is_sale_eligible, state, created_at, last_updated_at. Primary Key(id)
- **Foreign key:** product(category_id) refers category(id), indexed automatically by InnoDB.
- **Cardinality:** Product–Category m:1.
- **Elasticsearch index `products`:** id, name and description (text, English analyser), categoryName (text), categoryId (long), price (double), imageUrl (keyword, not indexed).

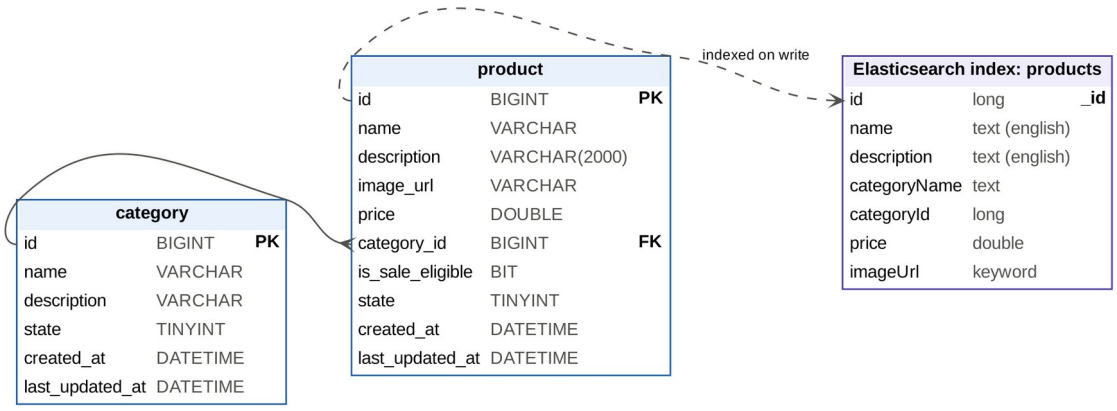


Figure 4.02: ER diagram: Product Catalogue service and search index

4.3 Order service (MySQL: orderservice_aug2026)

- **orders**: id, user_id, user_email, status (ENUM), total_amount, currency, payment_method (ENUM), payment_link, delivery address columns (full_name, line1, line2, city, state, pincode, phone_number), created_at, last_updated_at. Primary Key(id). Index idx_orders_user_created(user_id, created_at)
- **order_item**: id, order_id, product_id, product_name, unit_price, quantity, created_at. Primary Key(id)
- **order_status_history**: id, order_id, status, note, changed_at. Primary Key(id)
- **Foreign keys:** order_item(order_id) refers orders(id); order_status_history(order_id) refers orders(id).
- **Cardinality:** Order–OrderItem 1:m; Order–OrderStatusHistory 1:m.

The table is named `orders` because ORDER is a reserved SQL word. The composite index matches the order-history query exactly (`WHERE user_id = ? ORDER BY created_at DESC LIMIT 10`), so MySQL reads only the ten newest rows of one user instead of scanning and sorting the whole table (Section 5.4).

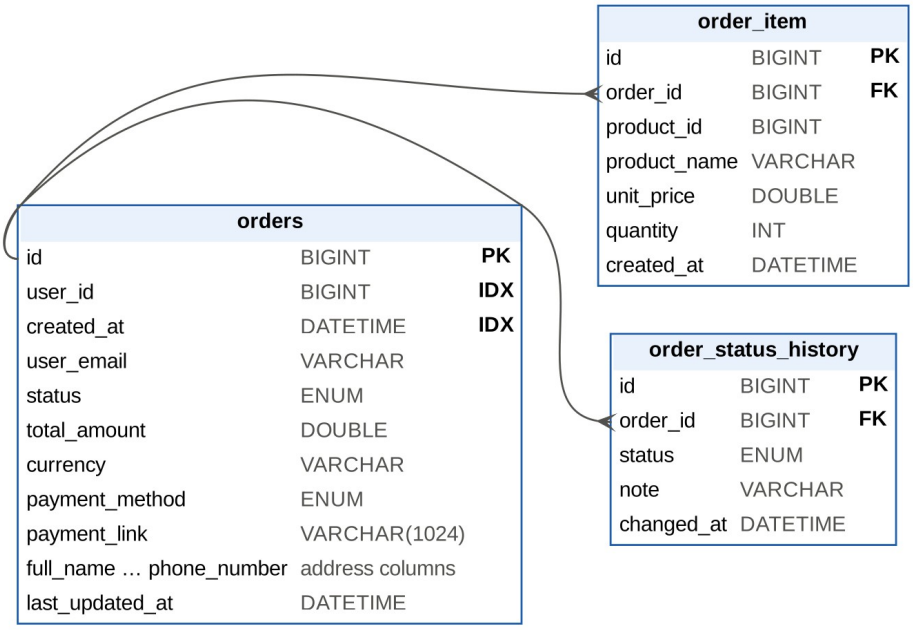


Figure 4.03: ER diagram: Order service

4.4 Payment service (MySQL), Cart (MongoDB) and Redis keys

- **payment**: id, order_id (unique, which makes event processing idempotent), user_id, user_email, amount (BIGINT, paise), currency, gateway, status, payment_link, gateway_reference (indexed; matched against webhooks), gateway_payment_id, receipt_number, failure_reason, created_at, last_updated_at, paid_at.
- **MongoDB carts**: `_id` = userId; `items[]` embedded {productId, productName, imageUrl, unitPrice, quantity}; updatedAt. One document per user is read and written as a unit, which suits a document store.
- **Redis**: `carts::<userId>` (cart JSON, TTL 30 min); `auth:token:<sha256>` (validated user, TTL 60 s); `request_rate_limiter.*` (token buckets).

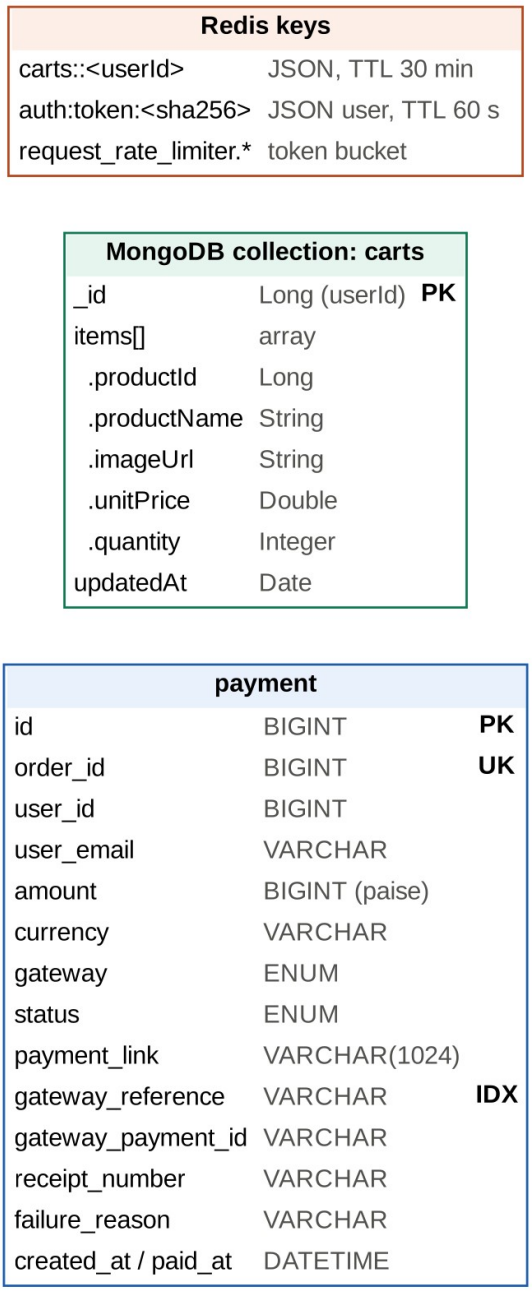


Figure 4.04: Payment table, cart document and Redis keys

4.5 Event schemas

Kafka messages are JSON, keyed by order id (or user id) so all events of one order land in the same partition and are consumed in order. The two central events are shown below.

```
order-events  (key = orderId)
{ "eventType": "ORDER_PLACED", "orderId": 42, "userId": 7,
  "userEmail": "asha@example.com", "status": "PAYMENT_PENDING",
  "totalAmount": 254997.0, "currency": "INR", "paymentMethod": "UPI",
  "items": [ { "productId": 3, "productName": "ZenBook 14",
```

```
               "unitPrice": 74999.0, "quantity": 1 } ],
  "timestamp": 1793034200000 }

payment-events  (key = orderId)
{ "eventType": "PAYMENT_SUCCEEDED", "orderId": 42, "paymentId": 9,
  "status": "SUCCEEDED", "amount": 25499700, "currency": "INR",
  "gateway": "RAZORPAY", "receiptNumber": "RCPT-20261026-9" }
```

5. Feature Development Process

The key feature chosen for this chapter is **product search** (PRD 2.3). It is the busiest read path of any store, and its quality directly affects sales. The chapter also covers the checkout and payment flow, which ties the services together, and the other optimisations that were measured.

5.1 Development process

1. **Baseline:** the original search matched the exact product name (`findByName`), so "laptop" found nothing called "ZenBook 14 Laptop".
2. **Functional fix:** a keyword search over name and description (`findByNameContainingIgnoreCaseOrDescriptionContainingIgnoreCase`) . It is correct but needs a full table scan, because a `LIKE '%term%'` pattern cannot use a B-tree index.
3. **Elasticsearch:** products are indexed into a denormalised `products` index on every create, update and delete. Search runs a `multi_match` query with field boosts and `fuzziness: AUTO`.
4. **Resilience:** MySQL stays the source of truth. If indexing fails, the product write still succeeds and the failure is logged; `POST /search/reindex` rebuilds the index in batches of 1,000. With Elasticsearch switched off, the MySQL query is used automatically.
5. **Measurement:** 200,000 products were seeded and both implementations were benchmarked with the same query mix.

5.2 Request flow

Figure 5.01 traces a search request. The gateway classifies `/search` as public, applies the per-client rate limit and forwards to an instance of `productservice` found through Eureka. `SearchController` (controller) validates paging, `SearchService` (service) chooses the engine and runs the query, and the controller maps `Page<Product>` to `Page<ProductDto>` (view).

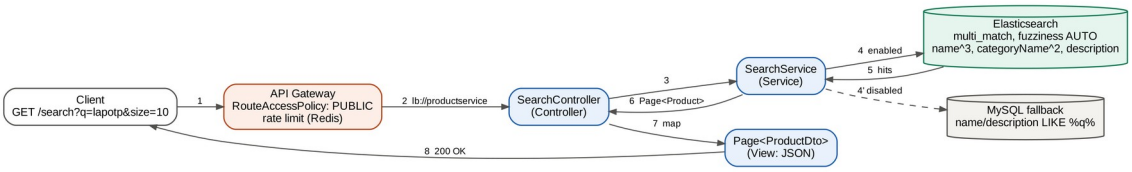


Figure 5.01: Search request flow through the MVC layers

API request and response:

```
GET /search?q=lapotp&page=0&size=10&sortBy=RELEVANCE
    (POST /search with a JSON body also works)

200 OK
{ "content": [ { "id": 1874, "name": "Rivo Laptop Ultra 887",
                 "price": 64999.0,
                 "description": "Rivo Laptop Ultra 887 with fast charging ...",
                 "category": { "id": 1, "name": "Seed Laptops" } }, ... ],
  "page": { "size": 10, "number": 0,
            "totalElements": 9987, "totalPages": 999 } }
```

Service logic (SearchService):

```
NativeQuery nativeQuery = NativeQuery.builder()
        .withQuery(q -> q.multiMatch(m -> m
                .query(query)
                // boost matches in name and category
                .fields("name^3", "description", "categoryName^2")
                .type(TextQueryType.BestFields)
                .fuzziness("AUTO")))        // tolerate 1-2 character typos
        .withPageable(pageable)
        .build();
SearchHits<ProductDocument> hits =
        elasticsearchOperations.search(nativeQuery, ProductDocument.class);
```

The misspelled query "lapotp" returns 9,987 laptops from Elasticsearch and none from the MySQL fallback. That gap in result quality matters as much as the speed difference shown below.

5.3 Checkout and payment flow

Checkout (Figure 5.02) shows how the services cooperate. The synchronous part is limited to what the customer must wait for: reading the cart, re-pricing each item at the current catalogue price, and saving the order. Everything else happens through Kafka. The ORDER_PLACED event is published only after the database transaction commits (`TransactionSynchronization.afterCommit`), so consumers never see an order that was rolled back. The cart is also cleared after the commit. The Payment service creates the payment link, and the customer receives it by email and on the order. A signed webhook from the gateway completes the payment and moves the order to PAID.

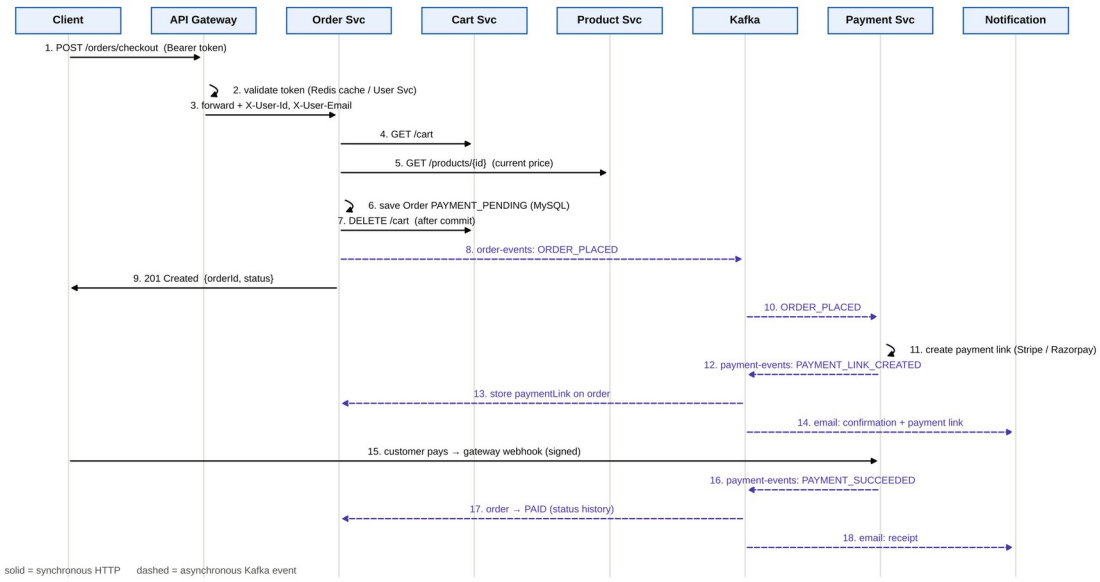


Figure 5.02: Checkout and payment sequence

Checkout request:

```
POST /orders/checkout      Authorization: Bearer <token>
{ "deliveryAddress": { "fullName": "Asha Kumar", "line1": "12 MG Road",
                       "city": "Bengaluru", "state": "KA",
                       "pincode": "560001", "phoneNumber": "9000000001" },
  "paymentMethod": "UPI" }

201 Created
{ "id": 42, "status": "PAYMENT_PENDING", "totalAmount": 254997.0,
  "currency": "INR", "paymentMethod": "UPI", "items": [ ... ],
  "createdAt": "2026-10-26T17:20:11.000+00:00" }
```

Reliability details that came out of testing:

- **Idempotency:** `payment.order_id` is unique, and `markSucceeded` returns early for an already-successful payment, so Kafka redeliveries and gateway webhook retries cannot create duplicate payments or emails.
- **Webhook security:** Stripe events are verified with `Webhook.constructEvent` and Razorpay events with an HMAC-SHA256 signature check. A test sends a forged signature and asserts that the payment stays PENDING.
- **Graceful degradation:** if the Product service is unreachable, Cart and Order answer 503 ("retry") instead of an opaque 500. A failure to publish a cart analytics event never fails the cart update.

- **Immediate logout:** the gateway caches validated tokens for 60 s. A logout or password reset publishes `session-revoked`, which evicts the cache entry at once. The end-to-end test confirms the old token is rejected within one second.

5.4 Performance optimisation and benchmarks

All benchmarks ran on the Docker Compose stack: 4 vCPU and 16 GB RAM, with every service and database on one machine. A load generator (`scripts/bench.py`) used 10 concurrent keep-alive clients, discarded a warm-up and reported throughput with p50/p95/p99 latency. Data sets were generated reproducibly (`scripts/seed_products.py`: 200,000 products in 20 categories; `scripts/seed_orders.py`: 300,000 orders for 3,000 users with 600,000 items). Raw results are kept in `docs/results`. Table 5.01 lists the results and Figures 5.03 and 5.04 compare them.

Table 5.01: Benchmark results (10 concurrent clients)

Scenario	Variant	p50 (ms)	p95 (ms)	Throughput (req/s)
Product search, 200k products	MySQL LIKE	1,624.0	2,404.7	5.9
	Elasticsearch	40.7	94.5	212.4
Order history, 300k orders	No index	827.9	1,185.2	11.6
	Composite index	21.4	34.1	445.7
Order history, item loading	N+1 (12 queries/page)	21.4	34.1	445.7
	@BatchSize (3 queries/page)	13.9	23.7	668.6
Cart read	MongoDB only	5.15	10.05	1,752.1
	Redis cache (warm)	3.55	7.32	2,404.8
Gateway authentication	Validate on every call	21.5	45.7	405.9
	Cached in Redis (60 s)	11.1	18.6	839.4

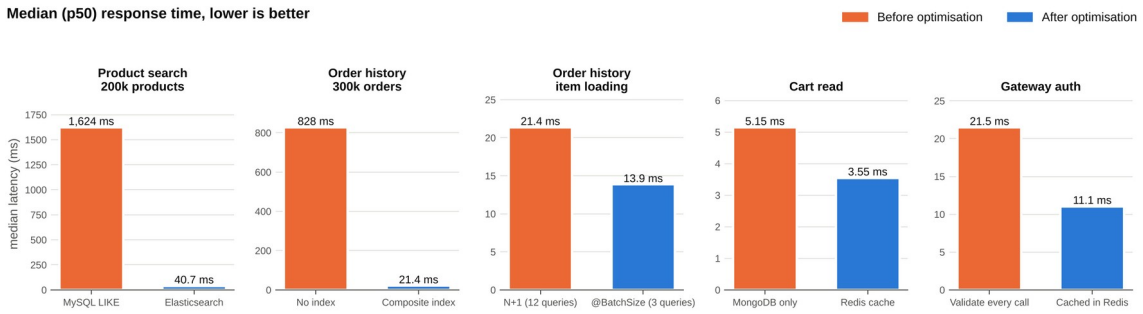


Figure 5.03: Median response time before and after each optimisation

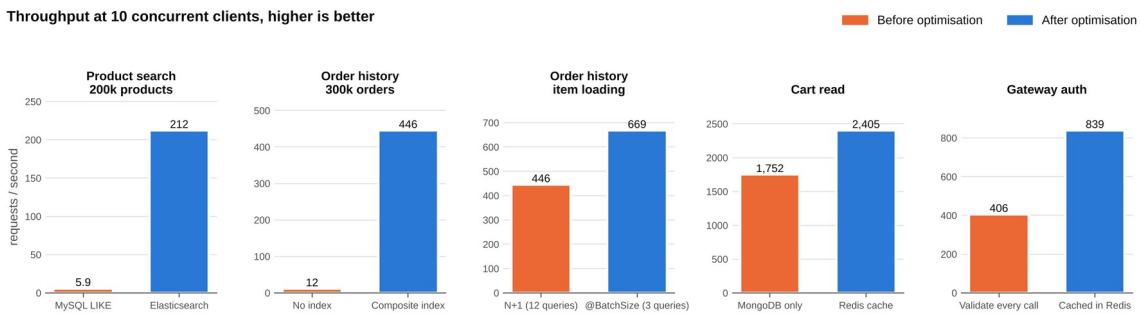


Figure 5.04: Throughput before and after each optimisation

Search: 40× lower latency. Elasticsearch answers from an inverted index whose size depends on the matching terms, not on the number of products. The MySQL fallback has to scan all 200,000 rows twice per request, once for the page and once for the total count. Elasticsearch served 212 requests per second against 5.9 for MySQL. Re-indexing all 200,000 products took 42 s.

Indexing: 39× lower latency. Without an index, `EXPLAIN` shows a full scan of 297,505 rows plus a filesort for every history page. With `idx_orders_user_created(user_id, created_at)` the plan becomes a `ref` lookup of about 94 rows read in index order ("Backward index scan"), with no sort.

N+1 removal: 35% lower latency and 50% more throughput. Lazy loading the items of 10 orders issued 10 extra SELECTs per page, 12 in total. MySQL's `Com_select` counter was measured over 100 requests. `@BatchSize(size = 50)` loads all items with one `WHERE order_id IN (...)` query (3 statements per page). Because the database runs on the same host, each round trip here costs well under a millisecond. On AWS, where every RDS round trip crosses the network, the saving per removed query is larger. Figures are the median of three runs of 2,000 requests each.

Caching. The Redis cart cache cut median latency by 31%, from 5.15 ms to 3.55 ms. The gain is modest because MongoDB already serves a primary-key lookup quickly, and most of the remaining time is HTTP and JSON work. The cache matters most under load, when it keeps reads off the database. The gateway's token cache halved latency and doubled throughput, because a cache hit skips a whole network hop to the User service plus a session lookup and a signature check. The trade-off is that a revoked token could stay valid for up to the cache TTL, which is why revocation events evict entries directly.

5.5 Testing

Table 5.02: Automated tests

Service	Tests	What they cover
User / Auth	8	Hashing, duplicate signup, admin role, login/validate/logout, unique tokens, password reset revokes sessions, silent reset for unknown email
Product Catalogue	10	Controller (MockMvc and direct), category reuse by name, keyword search and sort, validation, delete
Cart	6	Add/merge, quantity limits, unknown product, Kafka failure tolerance
Order	6	Checkout at current price, empty cart, payment events and duplicates, state machine, ownership
Payment	6	Paise conversion, idempotency, receipts, refund flag, Stripe without secret, Razorpay signature (valid and forged)
Notification	4	Context load, email templates
API Gateway	3	Access rules for public, user and admin routes
Service Discovery	1	Context load
End-to-end	58 checks	Every PRD flow through the gateway on the full Docker stack (`scripts/e2e-smoke.sh`)

Unit and integration tests run with `mvn package` and need no infrastructure: MySQL is replaced by H2 in MySQL mode and Kafka producers are mocked. The end-to-end script uses only `curl` and `jq` against the running stack. It found two real defects during development. First, the gateway token cache delayed logout. Second, a consumer subscribed before its topic existed could miss the first revocation. Both were fixed (Section 5.3).

6. Deployment Flow

Every service is packaged as an executable Spring Boot jar inside a small Docker image built from `eclipse-temurin:17-jre`. The JVM is started with `-XX:MaxRAMPercentage=75` so it sizes its heap to the container. All configuration (database URLs, Kafka brokers, secrets, feature flags) comes from environment variables, so one image runs unchanged on a laptop and on AWS.

6.1 Local and test deployment

```
./build-all.sh                 # builds all 8 services and runs the 44 tests
docker compose up -d --build   # MySQL, MongoDB, Redis, Kafka (KRaft),
                               # Elasticsearch + all services
./scripts/e2e-smoke.sh         # 58 end-to-end checks through the gateway
```

Compose health checks start the services only after MySQL, Kafka and Elasticsearch are ready. Kafka runs in KRaft mode, so ZooKeeper is not needed. The stack uses about 6 GB of memory.

6.2 AWS deployment

Figure 6.01 shows the production layout, which maps each HLD component to a managed AWS service (Table 6.01).

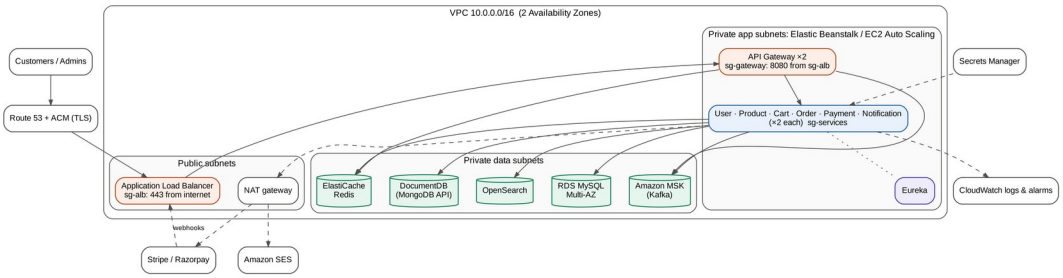


Figure 6.01: AWS deployment architecture

Table 6.01: HLD components on AWS

Component	AWS service	Configuration
Load balancer	Application Load Balancer (ELB)	Public subnets; TLS certificate from ACM; forwards only to the gateway
Compute (EC2 / managed infra)	Elastic Beanstalk on EC2 Auto Scaling	One environment per service; ≥ 2 instances across 2 AZs; scale on CPU
Network	VPC 10.0.0.0/16	Public (ALB, NAT), private app, and private data subnets in 2 AZs
MySQL	Amazon RDS for MySQL 8, Multi-AZ	One database per service; automated backups
Cache	Amazon ElastiCache for Redis	Cart cache, token cache, rate limits
MongoDB	Amazon DocumentDB	`MONGODB_URI` for the cart service

Component	AWS service	Configuration
Kafka	Amazon MSK	3 brokers across 3 AZs, replication factor 3
Search	Amazon OpenSearch Service	`ELASTICSEARCH_URIS`; rebuilt via `/search/reindex`
Email	Amazon SES (SMTP)	`SMTP_HOST=email-smtp.<region>.amazonaws.com`
Secrets	Secrets Manager / SSM Parameter Store	JWT key, DB passwords, gateway keys, SMTP credentials
Monitoring	CloudWatch	Logs, CPU/latency alarms, auto-scaling triggers

6.3 VPC and security groups

Only the load balancer is reachable from the internet. Application instances and data stores sit in private subnets and reach external APIs (Stripe, Razorpay, SES) through a NAT gateway. Security groups allow each tier to accept traffic only from the tier in front of it (Table 6.02). This is also what makes it safe for services to trust the `X-User-*` headers: only the gateway can reach them, and the gateway strips any copies a client sends.

Table 6.02: Security groups

Security group	Inbound from	Ports
sg-alb	0.0.0.0/0	443 (80 redirects to 443)
sg-gateway	sg-alb	8080
sg-services	sg-gateway, sg-services	8081–8086, 8761
sg-rds	sg-services	3306
sg-docdb	sg-services	27017
sg-redis	sg-gateway, sg-services	6379
sg-msk	sg-gateway, sg-services	9092 / 9094 (TLS)
sg-opensearch	sg-services	443

6.4 Release steps with Elastic Beanstalk

1. Create the VPC, subnets and security groups. Then create RDS, ElastiCache, DocumentDB, MSK and OpenSearch in the data subnets.
2. Store secrets in Secrets Manager and reference them as Beanstalk environment properties.
3. Build with `./build-all.sh` (tests must pass), then for each service run `eb init` and `eb create <service>-prod` with the VPC settings. The gateway gets the public ALB and internal services get none.
4. Set environment properties per service, for example `DB_URL`, `KAFKA_BOOTSTRAP_SERVERS`, `EUREKA_URL`, `PAYMENT_GATEWAY=RAZORPAY` and the gateway keys.

5. Deploy new versions with `eb deploy` (rolling deployment keeps capacity during the update).
6. Point Stripe/Razorpay webhooks at `https://<domain>/stripeWebhook` and `/payment/webhook/razorpay`, run `POST /search/reindex` once, and run the end-to-end script against the environment.

For a low-cost demonstration environment, Kafka, MongoDB and Elasticsearch can run on one EC2 instance with the same `docker-compose.yml`, with only RDS and ElastiCache managed. MSK, OpenSearch and DocumentDB are the most expensive parts of the fully managed layout.

7. Technologies Used

Table 7.01: Technology stack

Technology	Version	Used for
Java	17	Language for all services
Spring Boot	3.3.3 / 3.5.x	Web, data access, configuration, testing
Spring Cloud (Netflix Eureka, Gateway, LoadBalancer)	2023.0.3 / 2025.0.0	Discovery, API gateway, client-side load balancing
Spring Data JPA + Hibernate	6.5	MySQL persistence, paging, batch fetching
MySQL	8.0	Users, products, orders, payments
MongoDB	7.0	Carts
Redis	7.2	Cart cache, token cache, rate limiting
Apache Kafka (KRaft)	3.8.0	Event streaming between services
Elasticsearch	8.15.3	Product search
JJWT / Spring Security (BCrypt)	0.12.6	Token signing and password hashing
Stripe Java / Razorpay Java	32.1.0 / 1.4.8	Payment links and webhooks
JUnit 5, Mockito, MockMvc, H2	–	Unit and integration tests
Docker / Docker Compose	–	Packaging and local stack
AWS (ELB, Beanstalk, RDS, ElastiCache, MSK, OpenSearch, SES)	–	Production deployment

7.1 Spring Boot and Spring Cloud

Spring Boot builds stand-alone production applications with an embedded web server, auto-configuration and externalised configuration. Every service here is a Spring Boot application, and features such as caching (`@Cacheable`), transactions (`@Transactional`) and Kafka listeners (`@KafkaListener`) are declarative annotations. Spring Cloud adds the distributed-systems pieces: Netflix Eureka for service registration, Spring Cloud LoadBalancer behind `@LoadBalanced` clients, and Spring Cloud Gateway, a reactive gateway built on Project Reactor.

Real-world use: Spring is widely used for enterprise Java backends in banking, insurance, telecom and retail. Eureka was created and open-sourced by Netflix for its own streaming platform.

7.2 MySQL with Spring Data JPA

MySQL is a relational database with ACID transactions. It suits data with fixed structure and strong integrity rules, such as users, orders and payments. Spring Data JPA derives queries from method names (for example `findByUserIdOrderByCreatedAtDesc`) and

supports paging and sorting. Hibernate's fetch strategies (lazy loading, `@BatchSize`) and indexes were the main tuning levers in this project.

Real-world use: MySQL is one of the most widely deployed open-source databases and is commonly used for transactional web applications. On AWS it is offered as a managed service through Amazon RDS and Aurora.

7.3 MongoDB

MongoDB stores JSON-like documents with flexible schemas. A shopping cart, a user id with a variable list of items, maps naturally to one document that is read and written as a unit, with no joins.

Real-world use: Document databases are typical for catalogues, content management, user profiles and session-like data whose shape changes often. Amazon DocumentDB offers a MongoDB-compatible managed service.

7.4 Redis

Redis is an in-memory key-value store with sub-millisecond operations, expiry (TTL) and atomic operations. Here it serves three purposes: a cart cache, a cache of validated tokens at the gateway, and token buckets for rate limiting (Spring Cloud Gateway's `RedisRateLimiter` updates them atomically with a Lua script, so the limit holds across gateway instances).

Real-world use: Redis is used for caching, sessions, leaderboards, rate limiting and queues by many large web services; Twitter, GitHub and Stack Overflow have described using it.

7.5 Apache Kafka

Kafka is a distributed, partitioned, replicated commit log. Producers append events to topics, and each consumer group reads them at its own pace and can replay them. Events with the same key go to the same partition, which preserves per-order ordering here. Kafka gives the system its resilience: if the Notification service is down, emails are sent when it comes back, and adding a new consumer such as analytics needs no change to the producers. Kafka 3.8 runs in KRaft mode, without ZooKeeper.

Real-world use: Kafka was developed at LinkedIn for activity streams and is used for order pipelines, payments, log aggregation, change-data-capture and real-time analytics. Amazon MSK provides it as a managed service.

7.6 Elasticsearch

Elasticsearch is a distributed search engine built on Apache Lucene. It analyses text (tokenising, lower-casing, stemming with the English analyser) into an inverted index, ranks results by relevance (BM25), and supports fuzzy matching for typos. The product index is denormalised so that searches never touch MySQL.

Real-world use: Elasticsearch powers site search, log analytics (the ELK stack) and observability. Wikipedia's search and GitHub's earlier code search were built on it. Amazon OpenSearch Service is the managed AWS offering.

7.7 JWT, BCrypt and gateway security

JSON Web Tokens (RFC 7519) carry signed claims (user id, roles, expiry) that a server can verify without a database lookup. This project also stores sessions so tokens can be revoked. Passwords are hashed with BCrypt, a deliberately slow, salted algorithm recommended by OWASP for password storage. The gateway centralises authentication and authorisation so individual services stay simple.

Real-world use: Token-based authentication with a gateway or identity provider is the norm for mobile apps and single-page web apps, and is used by OAuth 2.0 and OpenID Connect providers.

7.8 Stripe and Razorpay

Both providers host the payment page, so card and UPI details never reach this system, which greatly reduces PCI-DSS scope. Payment links are created through their SDKs, and the result comes back through signed webhooks, which the Payment service verifies before changing any state.

Real-world use: Stripe serves businesses internationally; Razorpay is widely used in India and supports UPI, net banking, cards and wallets, matching PRD 5.1.

7.9 Docker and AWS

Docker packages each service with its runtime into an image that behaves the same everywhere. Docker Compose runs the whole system, including five data stores, with one command for development and testing. AWS provides the managed equivalents for production (Chapter 6).

Real-world use: Containers on managed cloud platforms are the standard way to deploy microservices; Elastic Beanstalk, ECS and EKS are common choices on AWS.

8. Conclusion

The project delivers a working, tested and measured e-commerce backend that implements every requirement of the PRD with the architecture of the HLD. Eight Spring Boot services cooperate through REST and Kafka, each with the data store that fits its access pattern. The complete purchase journey, from signup through search, cart, checkout, payment, receipt and delivery tracking, is verified automatically end to end.

8.1 Key takeaways

- **Choose the store for the access pattern:** relational tables for orders and payments, a document per cart, an inverted index for search and an in-memory cache for hot reads. The largest gains (40× for search, 39× for order history) came from these choices and from one well-chosen index, not from code tuning.
- **Asynchronous events decouple services but need discipline:** publish after commit, key events by aggregate id, make consumers idempotent, and model state explicitly (the order state machine).
- **Measure before optimising:** the N+1 problem was invisible in functional tests and obvious once statements were counted. The cart cache gave less than expected because MongoDB was already fast.
- **Security is a design concern:** server-side sessions make logout real, caching tokens requires an invalidation path, webhooks must be verified, and secrets belong in the environment. One credential was found committed in the original code and removed.
- **Automated end-to-end tests catch integration bugs** that unit tests cannot, such as topic-creation timing and cache invalidation.

8.2 Practical applications

The same building blocks apply well beyond retail. Food-delivery and ride-hailing apps use the same order state machine and event flow. Ticketing and travel booking need the same payment idempotency and webhook handling. Online-education platforms (course catalogue, enrolment and payment) map almost one to one onto these services. Any business that expects uneven traffic can use the gateway, caching and independent scaling patterns.

8.3 Limitations and cost

- **Social login** (PRD 1.1 mentions social media profiles) is not implemented. Email/password accounts are. Spring Security OAuth2 Login with Google is the natural extension.
- **Kong** (named in the HLD) was replaced by Spring Cloud Gateway to keep the stack in Java. The access rules are isolated in one class and could be moved to Kong plugins.
- **Inventory and stock reservation** are not modelled, so overselling is possible. A stock service with reservations on ORDER_PLACED would fix this.
- **Refunds** are flagged (REFUND_PENDING) but not executed automatically; a scheduled reconciliation job against the gateway is the next step.
- **Event publishing** happens after commit but is not a full transactional outbox. If the process crashes between commit and publish, an event could be lost; an outbox table with a relay, or Debezium CDC, would close the gap.
- **Schema management** uses Hibernate `ddl-auto=update`; production should use Flyway or Liquibase migrations.
- **Benchmarks** ran on one machine. Absolute numbers will differ on AWS, although the relative gains should hold.
- **Cost:** the fully managed AWS layout is dominated by MSK, OpenSearch and DocumentDB. Smaller deployments can self-host those on EC2, or use serverless options (MSK Serverless, OpenSearch Serverless) that charge by use.

8.4 Future work

- Distributed tracing (OpenTelemetry) and metrics dashboards, plus circuit breakers (Resilience4j) on inter-service calls.
- Recommendations and analytics built on the existing `cart-events` stream.
- A CI/CD pipeline (GitHub Actions) that builds, runs the tests, publishes images and deploys to Elastic Beanstalk.
- Product images on Amazon S3 with CloudFront, and caching of popular product pages.

References

1. Spring Boot Reference Documentation, docs.spring.io/spring-boot, accessed 26 October 2026, VMware/Broadcom, "Spring Boot 3.3".
2. Spring Cloud Gateway Reference, docs.spring.io/spring-cloud-gateway, accessed 26 October 2026, VMware/Broadcom, "RequestRateLimiter GatewayFilter Factory" and "Global Filters".
3. Spring Cloud Netflix Reference, docs.spring.io/spring-cloud-netflix, accessed 26 October 2026, "Service Discovery: Eureka Clients".
4. Spring Data Elasticsearch Reference, docs.spring.io/spring-data/elasticsearch, accessed 26 October 2026, "Elasticsearch Operations".
5. Hibernate ORM 6 User Guide, docs.jboss.org/hibernate/orm/6.5, accessed 26 October 2026, "Fetching: Batch fetching".
6. Apache Kafka Documentation, kafka.apache.org/documentation, accessed 26 October 2026, Apache Software Foundation, "KRaft" and "Design".
7. Elasticsearch Guide 8.15, elastic.co/guide, accessed 26 October 2026, Elastic N.V., "Multi-match query" and "Fuzziness".
8. Redis Documentation, redis.io/docs, accessed 26 October 2026, Redis Ltd., "Redis as a cache".
9. MongoDB Manual, mongodb.com/docs/manual, accessed 26 October 2026, MongoDB Inc., "Data Modeling: Embedded data".
10. MySQL 8.0 Reference Manual, dev.mysql.com/doc/refman/8.0, accessed 26 October 2026, Oracle, "Optimization and Indexes", "EXPLAIN Output Format".
11. Stripe Documentation, docs.stripe.com, accessed 26 October 2026, Stripe Inc., "Payment Links" and "Check webhook signatures".
12. Razorpay Documentation, razorpay.com/docs, accessed 26 October 2026, Razorpay, "Payment Links API" and "Validate and Test Webhooks".
13. AWS Documentation, docs.aws.amazon.com, accessed 26 October 2026, Amazon Web Services, "AWS Elastic Beanstalk Developer Guide", "Amazon VPC security groups", "Amazon MSK", "Amazon ElastiCache".
14. OWASP Cheat Sheet Series, cheatsheetseries.owasp.org, accessed 26 October 2026, OWASP Foundation, "Password Storage Cheat Sheet" and "JSON Web Token Cheat Sheet".

15. M. Jones, J. Bradley, N. Sakimura, "JSON Web Token (JWT)", RFC 7519, IETF, 2015.
16. J. Kreps, N. Narkhede, J. Rao, "Kafka: a Distributed Messaging System for Log Processing", Proceedings of the NetDB Workshop, 2011.
17. M. Kleppmann, "Designing Data-Intensive Applications", O'Reilly Media, 2017.
18. C. Richardson, "Microservices Patterns", Manning Publications, 2018.
19. S. Newman, "Building Microservices", 2nd edition, O'Reilly Media, 2021.
20. E. Gamma, R. Helm, R. Johnson, J. Vlissides, "Design Patterns: Elements of Reusable Object-Oriented Software", Addison-Wesley, 1994.
21. Scaler Academy, Backend Specialisation course material (Spring Boot, JPA, Kafka, Docker, AWS modules), 2026.
22. Anthropic, Claude (AI assistant), claude.ai, used in October 2026 for implementation assistance, code review, benchmarking scripts and drafting of this report; all output was reviewed and verified by tests.